Q38. Debugging Pair Sum Code

(Premature Return Inside Outer Loop)

Register No.	192425231
Student Name	VANJINATHAN S

1. Problem Statement

The task is to analyse a brute-force implementation of the Pair Sum problem, which is supposed to find two indices i and j in an array such that arr[i] + arr[j] equals a given target value. The given implementation contains a logical placement error: the return None statement sits inside the outer for loop instead of after it, causing the function to terminate prematurely and report failure even when a valid pair exists later in the array.

2. Original (Buggy) Code

```
def pair_sum(arr, target):
    for i in range(len(arr)):
        for j in range(i+1, len(arr)):
            if arr[i] + arr[j] == target:
                return (i, j)
        return None   # ERROR: misplaced inside outer loop
```

3. Why the Bug Occurs — Step-by-Step Analysis

Consider the call pair_sum([2, 7, 11, 15], 26). The correct answer is the pair at indices (2, 3), since 11 + 15 = 26. Tracing the buggy code line by line:

- When i = 0, the inner loop runs j = 1, 2, 3. None of these sums (9, 13, 17) equal 26.
- Because no match was found, the inner for loop finishes normally (it is not exited via return).
- Control now falls to the very next line, which is return None. This line is indented at the SAME level as the inner for loop, meaning it belongs to the body of the outer loop, not to the function's final statement.
- As a result, the function returns None immediately after completing just the first outer iteration (i = 0) — it never proceeds to i = 1 or i = 2, where the true answer (2, 3) is located.

In short, the outer loop only ever completes one iteration. On i = 0, if a match is found the function returns it (correct); if not, the misplaced return None triggers, incorrectly declaring failure before the remaining outer iterations (i = 1, 2, ...) ever run. Valid pairs occurring later in the array are therefore never considered.

3.1 Execution Trace for arr = [2, 7, 11, 15], target = 26

i	j	arr[i]	arr[j]	Sum	Match?	Loop Action
0	1	2	7	9	No	continue inner loop
0	2	2	11	13	No	continue inner loop
0	3	2	15	17	No	inner loop ends
1	2	7	11	18	No	continue inner loop
1	3	7	15	22	No	inner loop ends
2	3	11	15	26	Yes	return (2, 3)

The trace shows that after the i = 0 pass (rows 1–3) fails to find a match, the buggy return None fires immediately — the algorithm never reaches i = 1 or i = 2 (rows 4–6), where the correct pair (2, 3) actually satisfies 11 + 15 = 26. This confirms that the true answer, found only at row 6, is never reached by the faulty version because the loop is aborted right after row 3.

4. The Fix — Correcting Indentation / Scope

The remedy is purely structural: the return None statement must be dedented so that it aligns with (i.e., is a sibling of) the outer for loop, not the inner one. This ensures None is only returned after BOTH loops have fully completed — meaning every possible index pair (i, j) with j > i has been checked and none matched the target.

- Keep return (i, j) inside the inner loop, guarded by the if condition — this allows an early, correct exit the moment a valid pair is found.
- Move return None to the same indentation level as the outer for statement, so it executes only after the outer loop is fully exhausted.

5. Avoiding Redundant Mirror Comparisons

Even after fixing the indentation, the nested loop must avoid redundant "mirror" comparisons — i.e., checking (i, j) and later re-checking the equivalent pair (j, i), or comparing an element with itself (i, i). This is handled by starting the inner loop at j = i + 1 rather than j = 0:

- j = 0 would allow i and j to take the same index (i == j), an invalid self-pair.
- j starting from 0 would also re-examine pairs already covered when i had a smaller value (e.g., (0,2) and later (2,0) represent the same unordered pair), doubling unnecessary work.
- By restricting j to range(i + 1, n), each unordered pair of indices is generated and tested exactly once, halving the number of comparisons compared to a naive double loop over the full range, while still preserving pure brute-force (no sorting, hashing, or auxiliary structures).

6. Corrected Final Code

```
def pair_sum(arr, target):
    """
    Brute-force Pair Sum search.
    Returns index pair (i, j) such that arr[i] + arr[j] == target,
    or None if no such pair exists.
    Time Complexity : O(n^2)  -- nested loop, no early exit bug
    Space Complexity: O(1)   -- no extra data structures used
    """
    n = len(arr)

    # Outer loop selects the first element of the pair
    for i in range(n):
        # Inner loop starts at i+1 to avoid re-checking earlier
        # elements and to avoid comparing an element with itself
        # (this also removes redundant mirror comparisons such as
        # checking (i, j) and later (j, i) again)
        for j in range(i + 1, n):
            if arr[i] + arr[j] == target:
                # Valid pair found -- safe to return immediately
                # because we are still INSIDE the inner loop
                return (i, j)
        # NOTE: 'return None' is NOT placed here.
        # Placing it here would end the function after just
        # ONE pass of the outer loop, before every combination
        # has been checked.

    # Only reached after BOTH loops have completed fully,
    # i.e. every possible pair has been examined and failed.
    return None
```

7. Sample Runs

```
>>> pair_sum([2, 7, 11, 15], 9)
(0, 1)
```

```
>>> pair_sum([2, 7, 11, 15], 26)
(2, 3)

>>> pair_sum([2, 7, 11, 15], 100)
None
```

8. Time and Space Complexity Analysis

- Time Complexity: O(n²). The outer loop runs n times and, for each i, the inner loop runs up to (n - i - 1) times. Summing this over all i gives n(n-1)/2 comparisons in the worst case (no pair found), which is still O(n²) — the j = i+1 optimisation removes the constant-factor redundant/mirror checks but does not change the asymptotic class, since brute force is preserved by design.
- Space Complexity: O(1). Only a fixed number of scalar variables (i, j, n) are used; no auxiliary array, hash map, or recursion stack is introduced.
- Best Case: O(1) if the very first pair checked (i=0, j=1) already satisfies the target sum.
- Worst Case: O(n²) when no valid pair exists, forcing every one of the n(n-1)/2 combinations to be examined before returning None.

9. Conclusion

The original implementation failed because a single line — return None — was placed one indentation level too deep, binding it to the outer loop's body rather than to the function's overall control flow. This caused the search to abandon the array after only the first outer iteration, silently skipping every later valid pair. Correcting the indentation, combined with starting the inner loop at i + 1 to eliminate self-comparisons and mirrored duplicate checks, produces a brute-force O(n²) solution that is both logically correct and free of redundant work, while remaining faithful to the original brute-force approach requested.